Persistent Soft-Plan Decoding

A Block-Causal Probabilistic Proposal Architecture for Language Modeling

Lucka Barbeau

MyLocal_LLM research project • https://github.com/Luckabarbeau/MyLocal_LLM

Technical report v0.1 14 October 2026 Experimental architecture series 0080

Status. This is a dated, reproducibility-oriented project research note, not a peer-reviewed paper or a state-of-the-art claim. Results below are from local 7M-class CPU experiments reported through 2,500 additional expanded-corpus training steps. The separate approximately 500M-parameter integration is *not* evaluated here for quality or production throughput.

Abstract

We describe a block-causal language-model architecture that predicts a fixed group of K future-token distributions from one causal Transformer boundary state, converts all K distributions into differentiable soft token embeddings, and supplies them as a *persistent bidirectional plan prefix* to a lightweight causal decoder. The decoder selects tokens sequentially, conditioning on both the original soft plans and the tokens already committed within the block. A residual correction to the backbone's shared vocabulary logits yields normalized next-token distributions; joint supervision trains both direct future proposals and corrected predictions. The architecture represents a tractable factorization of a conditional K-token distribution without constructing an explicit V^K joint vocabulary.

Preliminary experiments on a roughly 7M-class model ($K = 4$, 2,048-token vocabulary) show improving held-out cross-entropy on expanded training data, measurable reliance on cross-position soft-plan inputs, and particularly pronounced sensitivity to future-position plans at the first output position. At step 2,500, full-plan cross-entropy is 4.1840 nats/token; neutralizing all future plans for the first target increases its loss by 0.2908 nats/token. These are *frozen-model interventions*, not causal estimates of an independently retrained architecture's advantage. A commitment-stride comparison on 256 reference tokens favors preserving the four-token plan ($S = 4$: 3.96349 nats/token) over replanning every one or two tokens ($S = 1$: 4.02431, $S = 2$: 4.02332), while free-running generation remains markedly incoherent. We outline precise limitations, code-level reproduction, and next experiments required to test planning, generalization, scaling, and performance.

Contents

1 Motivation, scope, and hypotheses **3**
1.1 Relation to existing work . . . 3

2 Architecture and causal information flow **3**
2.1 Sequence alignment and proposal boundaries . . . 3
2.2 Shared multi-horizon proposal head . . . 4
2.3 Differentiable probability-derived PLAN embeddings . . . 4
2.4 A 2K-position prefix Transformer . . . 4
2.5 Residual output and conditional block factorization . . . 5

3 Training objective and implementation contracts **6**
3.1 Joint loss and gradient paths . . . 6
3.2 Causal safety and block scheduling . . . 6
3.3 Parameter and computational considerations . . . 7

4 Exploratory experimental setup **7**
4.1 Two successive dataset regimes . . . 7
4.2 Measurements and terminology . . . 8

5 Preliminary empirical results **8**
5.1 Improving held-out final likelihood on expanded data . . . 8
5.2 What the frozen cross-position ablations actually change . . . 9
5.3 Future-plan contribution is strongest before commitment . . . 9
5.4 The persistent earlier soft proposal matters after commitment . . . 10
5.5 Free-running generation: predictive CE and fluency diverge . . . 10

6 Receding-horizon experiment: commitment stride **11**
6.1 Hypothesis . . . 11
6.2 Paired causal-inference scoring and work . . . 11
6.3 Interpretation and free-running comparison . . . 11

7 What is established, what is not, and next experiments **12**
7.1 Supported findings . . . 12
7.2 Open questions and threats to validity . . . 12
7.3 Proposed next experiments . . . 13

8 Conclusion **13**

A Source-code map and reproducibility commands **13**
A.1 PowerShell: fixed cross-position ablation . . . 14
A.2 PowerShell: paired commitment-stride evaluation . . . 14
A.3 PowerShell: qualitative generation by stride . . . 14
A.4 Run provenance and data availability . . . 15

B Earlier restricted-corpus K=4 result (not matched) **15**

1 Motivation, scope, and hypotheses

Conventional autoregressive (AR) language modeling uses a causal representation $h_t = f_\theta(x_{\le t})$ to predict one next-token distribution. A sequence of future tokens is generated by repeating this operation after each commitment. The present work explores a different allocation of computation: the expensive causal backbone first describes several plausible next positions, while a much smaller module learns how those uncertain predictions should be reconciled into a valid sequence.

The initial intuition is illustrated by the hypothetical phrase pair *New York* versus *San Francisco*: a plausible first token constrains the next one, but an uncertain next-token distribution might benefit from representations of possible later continuations. The architecture does not enumerate such phrases or create an explicit compound tokenizer. Instead, it gives a learned causal decoder access to all projected soft future marginals, including those corresponding to already committed positions.

The central research questions are:

1. Can the decoder use *future-position proposal representations* to improve earlier token probabilities while remaining strictly causal with respect to ground-truth targets?
2. Can the original soft proposals remain useful *after* the corresponding token has been committed, providing information complementary to the hard token embedding?
3. Does the mechanism improve free-running coherence, repetition behavior, likelihood, or efficiency relative to carefully matched AR and multi-token baselines?
4. How does the optimal proposal horizon K, commitment stride S, and decoder capacity change as the backbone grows?

These are hypotheses. The early measurements support a dependence on soft-plan representations; they do not yet prove higher-level semantic planning or superiority to matched AR systems.

1.1 Relation to existing work

The design is related to, but distinct from, several existing approaches. Multi-token prediction supervises multiple future horizons above a shared backbone [3]; multiple decoding heads also appear in speculative or tree-based acceleration systems such as Medusa [4]. Prefix and seq2seq attention masks that combine bidirectional context with causal generation have precedents, including UniLM [2]. Standard Transformers provide the underlying self-attention mechanism [1]. This work focuses on the particular *combination* of full soft future distributions, an explicitly bidirectional small PLAN prefix, a separate hard committed-token stream, persistent access to both throughout a block, and end-to-end corrected-token training. This description should not be treated as a comprehensive prior-art or patent-novelty assessment.

2 Architecture and causal information flow

2.1 Sequence alignment and proposal boundaries

Let $x_{\le t}$ denote the genuinely available context immediately before generation of token y_1. In the implemented training alignment, the ordinary source tokens are paired with their conventional shifted next-token labels; the backbone's proposal boundary for a block begins at the hidden state that has already processed the source token immediately preceding that block. In a training window, all source positions can be processed in a single causal trunk pass, but the proposal for a given block is formed *only* from that block's corresponding boundary state. Thus no target y_j from the proposed block is used to construct the plan.

For a proposal horizon K, one boundary state $h \in \mathbb{R}^d$ produces

$$L = [\ell_1, \dots, \ell_K] \in \mathbb{R}^{K \times V}, \qquad p_j = \mathrm{softmax}(\ell_j) \in \Delta^{V-1}, \tag{1}$$

where V is the vocabulary size. These p_j are *marginal horizon proposals conditioned on the same pre-block context*, not independently sampled tokens, and not an explicit joint probability table.

2.2 Shared multi-horizon proposal head

The implementation uses one shared nonlinear head and the backbone's existing (untied) vocabulary output projection W_{vocab}. Writing a_j for a learned horizon embedding, the implemented form is

$$u_1 = h, \tag{2}$$
$$u_j = h + \rho \tanh(hW_{\text{ctx}} + a_j)W_{\text{back}}, \quad j = 2, \ldots, K, \tag{3}$$
$$\ell_j = W_{\text{vocab}}(u_j), \tag{4}$$

with compatible tensor orientation understood. In the default research implementation, $\rho = 0.1$. Horizon one uses the same normalized boundary hidden state and vocabulary projection as a conventional one-step LM head. Later horizons share the small proposal MLP and the same vocabulary projection; there are no K independent vocabulary matrices.

2.3 Differentiable probability-derived PLAN embeddings

The entire proposal distribution enters the small decoder through an expectation over the original input embedding table $E \in \mathbb{R}^{V \times d}$:

$$z_j = p_j^\top E = \sum_{v=1}^{V} p_j(v) E_v \in \mathbb{R}^d. \tag{5}$$

This is a *compressed soft representation.* The decoder does *not* directly receive the complete V probabilities as a sequence of V inputs, and z_j need not preserve every detail of p_j because $d \ll V$. Gradients nevertheless flow through the expectation and the proposal softmax to the proposal logits, as well as into the input embeddings.

With learned projections W_P, W_D and learned position vectors r_j^P, r_j^D, the two streams are

$$P_j = W_P z_j + r_j^P, \quad j = 1, \ldots, K, \tag{6}$$
$$D_1 = b_{\text{BOS}} + r_1^D, \tag{7}$$
$$D_j = W_D E[y_{j-1}] + r_j^D, \quad j = 2, \ldots, K. \tag{8}$$

The distinction is essential: P_{j-1} *is never replaced by* $E[y_{j-1}]$ *after commitment.* The original predicted representation remains present in the PLAN stream, while the actual preceding token enters a separate DEC position. The PLAN and DEC embeddings use different learned input projections even though they refer to the same underlying input embedding table.

2.4 A 2K-position prefix Transformer

The small decoder concatenates all PLAN positions *before* all DEC positions:

$$X_{\text{small}} = [P_1, \ldots, P_K, D_1, \ldots, D_K] \in \mathbb{R}^{2K \times d_D}. \tag{9}$$

Its attention mask $A \in \{0, 1\}^{2K \times 2K}$ is specified by

$$A_{qk} = \begin{cases} 1, & q \le K,\ k \le K, \\ 1, & q > K,\ k \le K, \\ 1, & q > K,\ K < k \le q, \\ 0, & \text{otherwise.} \end{cases} \tag{10}$$

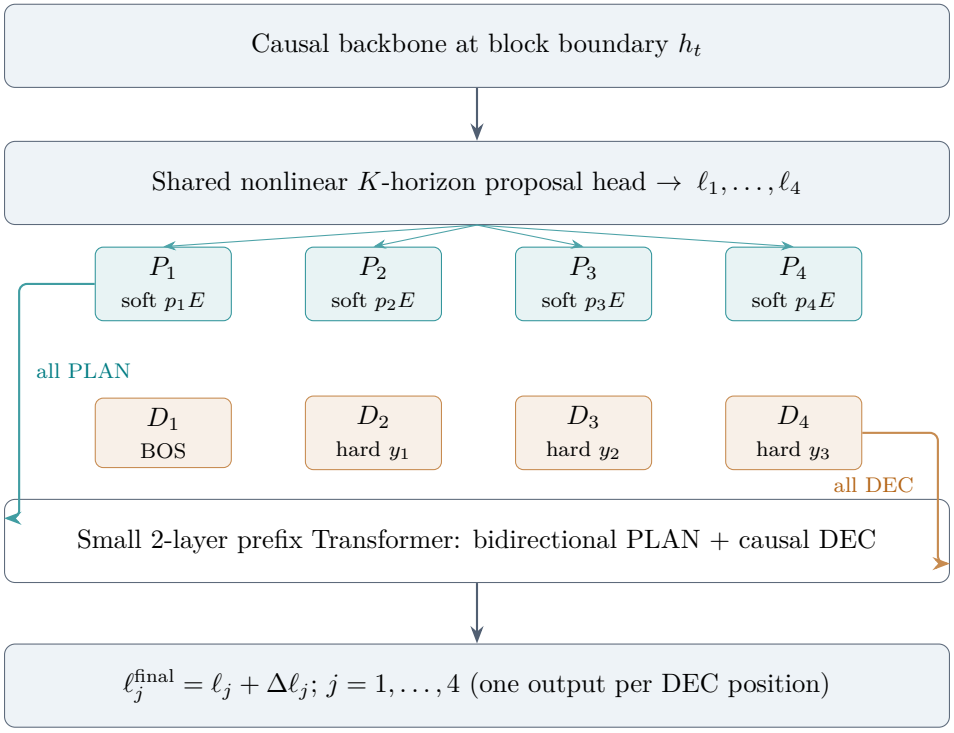


Figure 1: The implemented $K = 4$ layout. The full PLAN remains accessible during all four sequential token commitments. The diagram separates the soft plan from the shifted hard-token stream; arrows from all PLAN and DEC positions into the small decoder indicate allowed pathways, not equal attention weights.

The first K rows are fully bidirectional over the *predicted* PLAN positions and cannot read any DEC row. Every DEC row can read all PLAN positions and only its own or earlier DEC rows. The graph is block-causal because no PLAN input depends on a true token in the proposed block; all DEC inputs are shifted committed tokens.

The small stack uses dense self-attention with RMSNorm and SwiGLU feed-forward layers; the tested CPU decoder has two layers, width 128 and four attention heads, with no MoE inside the decoder. During teacher-forced training all DEC outputs are evaluated in one masked forward per bounded group of independent blocks. In cached inference, the PLAN prefix is processed once, its per-layer keys/values are retained, and each committed-token DEC position is appended sequentially.

2.5 Residual output and conditional block factorization

The small decoder outputs a position-specific correction $\Delta\ell_j$, using a projection back to the backbone's hidden width and the *same* vocabulary output projection as the proposal head:

$$\ell_j^{\text{final}} = \ell_j + \Delta\ell_j, \qquad q_j(\cdot \mid c, y_{<j}) = \text{softmax}(\ell_j^{\text{final}}). \tag{11}$$

A valid conditional probability for a complete block is then defined autoregressively by

$$Q_\Theta(y_{1:K} \mid c) = \prod_{j=1}^{K} q_j(y_j \mid c, y_{<j}, Z(c)), \qquad Z(c) = (z_1, \ldots, z_K). \tag{12}$$

This *implicit* joint sequence distribution is normalized without enumerating V^K combinations. It is not a learned explicit categorical distribution over V^K phrases. Because $Z(c)$ is computed from context alone, an unrestricted Bayes-optimal AR predictor conditioning on the same c cannot gain additional information from $Z(c)$; the potential benefit is *finite-model representation, optimization, or computation efficiency.*

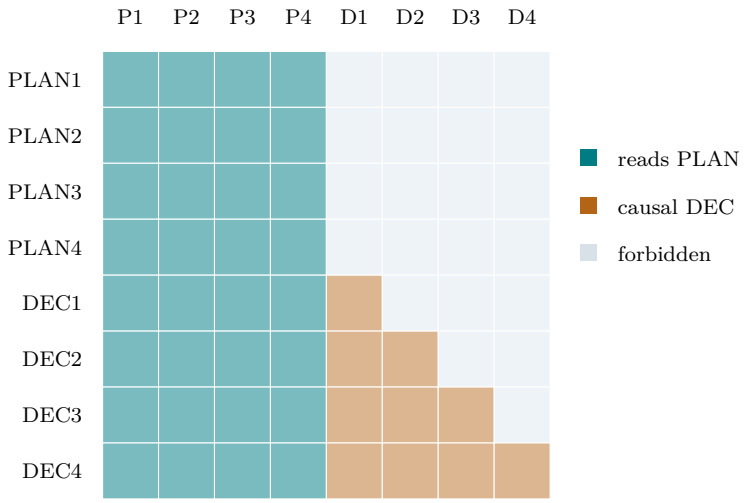


Figure 2: Exact attention permission structure for $K = 4$. Rows are queries and columns are keys. The forbidden PLAN←DEC region prevents targets from leaking into soft proposals through decoder self-attention.

3 Training objective and implementation contracts

3.1 Joint loss and gradient paths

Let $m_{bj} \in \{0, 1\}$ indicate that horizon j in block b corresponds to a real supervised token, and let $N_{\text{valid}} = \sum_{b,j} m_{bj}$. The actual objective is

$$\mathcal{L}_{\text{final}} = -\frac{1}{N_{\text{valid}}} \sum_{b,j} m_{bj} \log q_{bj}(y_{bj}), \tag{13}$$

$$\mathcal{L}_{\text{proposal}} = -\frac{1}{N_{\text{valid}}} \sum_{b,j} m_{bj} \log p_{bj}(y_{bj}), \tag{14}$$

$$\mathcal{L} = \mathcal{L}_{\text{final}} + \lambda \mathcal{L}_{\text{proposal}}, \qquad \lambda = 0.5 \text{ in the reported CPU experiment.} \tag{15}$$

Each real token counts *once* in the normalization; the two losses are not counted as two independent supervised examples. The final CE propagates gradients through the direct residual logits, the decoder, soft expectations pE, the proposal softmax, the shared embedding table, and the main causal boundary state. The auxiliary proposal CE provides direct training pressure at all horizons. An implementation with hand-written backward passes preserves both gradient routes.

3.2 Causal safety and block scheduling

Training enforces the following invariants:

- A block's proposals read only its *pre-block* causal boundary hidden state, never a later hidden state from the teacher-forced source window.
- PLAN positions cannot read DEC positions; DEC_j sees only BOS, earlier real target tokens, and all precomputed PLAN positions.
- Document boundaries, end-of-sequence handling, routed-prefix history starts, and invalid tail masks are preserved; unrelated documents are not silently concatenated as one proposal block.
- Main-model routed memory is selected using a strictly pre-window query. The original local, dilated, sparse, and learned retrieval heads remain in the main backbone.
- The main trunk is run once per training window. The bounded K-block groups reuse gathered causal boundary states, accumulate gradient contributions there, and perform one main-trunk backward pass.

These invariants distinguish the architecture from merely providing the decoder with the future teacher-forced sequence. Inference additionally updates the main causal KV state *only with committed tokens*, never provisional future predictions.

3.3 Parameter and computational considerations

The K proposal logits are full-vocabulary distributions but reuse the backbone output projection, so the proposal head does not instantiate K independent matrices of shape $d \times V$. The small decoder processes $2K$ positions, typically much shorter than the full active context. However, projecting and retaining all K full-vocabulary logits, evaluating probability expectations, and differentiating through two CE objectives can still incur significant bandwidth, memory, and synchronization overhead. Exact GPU throughput, allocation peaks, and parity must be measured: a *single trunk pass in training* does not automatically imply faster end-to-end inference or training than a matched standard AR model.

4 Exploratory experimental setup

4.1 Two successive dataset regimes

The CPU research experiment started with a compact two-source packed mixture, built using a preprocessing cap of 256 documents per source and an unchanged custom 2,048-token tokenizer. That restricted preparation contained only about 683,000 packed training tokens. The original $K = 4$ CPU run trained for 5,000 steps on this smaller preparation. It was subsequently *weight-only warm-started* on a separately prepared, much larger mixture capped at 50,000 documents per source, with a fresh AdamW optimizer state and sampling sequence. The larger dataset and held-out draw use distinct provenance; cross-dataset CE comparisons must therefore *not* be interpreted as matched validation learning curves.

The two sources used for these experiments are named `fine_edu` and `cosmopedia_v2`. Packing, document/EOS boundaries, tokenizer fingerprint, mixture manifest, validation sample digests, sampler metadata and source-checkpoint lineage are recorded or checked by the current research pipeline. Expanded-run step numbers begin at *zero* after the weight-only warm start and must not be confused with the original 5,000 optimizer steps.

Table 1: Reported experimental configuration for the expanded-corpus CPU checkpoint series. "Approximately 7M" is a model class, not an independently audited parameter count for this report.

Backbone	6 layers; width 192; MoE with 4 experts, top 2	Attention	local, dilated, sparse, retrieval
Context	256-token main memory horizon; 64-token targets	Vocabulary	2,048 tokens
Proposals	$K = 4$; shared head width 96; $\rho = 0.1$	Decoder	2 layers; width 128; 4 heads
Objective	$\mathcal{L}_{\text{final}} + 0.5\mathcal{L}_{\text{prop}}$	Data	two-source packed mixture, expanded cap 50k/source
Optimizer	AdamW; fresh state at warm start	LR	1×10^{-4} in expanded run
Batching	batch 16; accumulation 1; block group 2	Validation	16 deterministic batches of 16 rows
Execution	NumPy CPU, float32	Checkpoints	every 500 expanded steps

The primary *teacher-forced frozen-plan* validation uses the same fixed examples for each intervention: 16 batches of 16 rows, with up to 64 target tokens per row (at most 16,384 nominal token positions; the actual valid-token count can be smaller). The separate stride experiment intentionally uses only eight rows with up to 32 tokens each, totaling 256 scored tokens in the reported run.

4.2 Measurements and terminology

All CE values are negative log-likelihood in *nats per real target token*; lower is better. "Proposal CE" scores p_j against the true token at that horizon. "Final CE" scores q_j using teacher-forced correct earlier tokens unless explicitly identified as *causal-inference scoring*. The "decoder gain" is proposal CE minus final CE; it is not an independent information measure because the final model receives the correct earlier target tokens. "Frozen masking gain" means a change in final CE when some PLAN vectors are neutralized, without retraining model weights.

5 Preliminary empirical results

5.1 Improving held-out final likelihood on expanded data

Table 2: Reported expanded-corpus checkpoints, with the same held-out research-validation regime. The step-1,500 number is the mean of the four reported per-horizon validation CEs. Other values are from the frozen 0080K all-plan condition.

Expanded optimizer step	Final CE (nats/token)	Status
1,000	4.2909	Fixed all-plan evaluation
1,500	≈ 4.2443	Reported horizon means
2,000	4.2101	Fixed all-plan evaluation
2,500	**4.1840**	Fixed all-plan evaluation

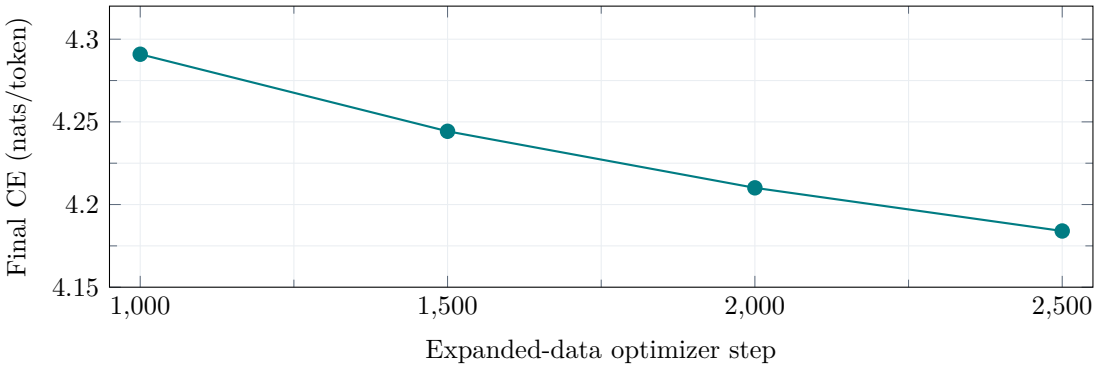


Figure 3: Held-out token likelihood continues improving after the weight-only warm start. This plot does not compare against a matched AR baseline.

At expanded step 2,000, the raw proposal and final losses by horizon show how much the sequential decoder changes the predictions: The final-position losses are nearly flat, while later direct proposals

Table 3: Expanded step-2,000 per-horizon teacher-forced validation. The proposal head sees the same pre-block context for all four positions; the decoder uses the real prior tokens for later positions.

Horizon	Proposal CE	Final CE	Decoder gain
P1	4.7710	4.2271	+0.5439
P2	5.8867	4.2035	+1.6832
P3	6.2546	4.1931	+2.0615
P4	6.4074	4.2167	+2.1907

become progressively less certain. The natural explanation includes the true earlier tokens made

available by teacher forcing; the table alone does not demonstrate multi-token reasoning or better free-running generation.

5.2 What the frozen cross-position ablations actually change

For each target horizon j, the 0080K evaluator holds fixed the main model, proposal logits and their additive residual-prior contribution, the source context, and the shifted *true* committed-token prefix. Only the *soft PLAN representations* are selectively replaced by a uniform-vocabulary expected embedding (with positional embeddings still present). The conditions are:

- **All:** $P_1, \ldots, P_K$ intact.
- **Own only:** retain only P_j; neutralize all other PLAN vectors.
- **Past + current:** retain $P_1, \ldots, P_j$; neutralize $P_{j+1}, \ldots, P_K$.
- **All except current:** retain every P_i except P_j.
- **Neutral:** neutralize all soft PLAN vectors while leaving direct logits and true shifted DEC inputs fixed.

The ablation loss penalty is $\Delta\,\mathrm{CE}_A = \mathrm{CE}_A - \mathrm{CE}_{\mathrm{all}}$. A positive penalty shows that the *trained frozen predictor* is sensitive to that PLAN pathway. It is not a valid decomposition of mutual information, and masking can introduce out-of-distribution representations that a separately retrained architecture would handle differently.

Table 4: Expanded-data frozen plan ablation: average final CE. Each column uses its own saved checkpoint and the same expanded held-out regimen.

Condition	Step 1,000	Step 2,000	Step 2,500
All plans	4.2909	4.2101	**4.1840**
Own only	4.5505	4.4813	4.4841
Past + current	4.3911	4.3061	4.2975
All except current	4.7384	4.6207	4.6528
Neutral plans	5.1664	5.0389	5.1612

Table 5: Frozen-plan loss penalties relative to all plans. "Other" includes both earlier and future PLAN slots; "future" is the only contrast isolating the removal of future slots at the decoder input.

Penalty	Step 1,000	Step 2,000	Step 2,500
Other-position plans: own only − all	+0.2596	+0.2711	**+0.3001**
Future plans: past/current − all	+0.1002	+0.0960	**+0.1136**
Own plan: all-except-own − all	+0.4475	+0.4106	+0.4688
All soft plans: neutral − all	+0.8755	+0.8288	+0.9772

The full-plan model's loss falls between steps 2,000 and 2,500, while some frozen masked conditions cease improving or worsen. This is consistent with increasing reliance on the learned PLAN pathway, but the increasing *gap* could partly reflect growing sensitivity to a distribution shift induced by neutralized input slots.

5.3 Future-plan contribution is strongest before commitment

The first position is especially informative because there are no preceding real tokens inside the block. At P1, the difference between *all* and *past + current* is precisely the sensitivity to future soft PLAN positions P2–P4; the shifted DEC input is only learned BOS.

A useful interpretation of the P1 step-2,500 result is that neutralizing P2–P4 raises the average correct-token NLL by 0.2908 nats, corresponding to a factor $e^{0.2908} \simeq 1.34$ in geometric-mean assigned

Table 6: Per-horizon sensitivity to neutralizing strictly future PLAN positions. By construction the P4 difference is exactly zero.

Prediction position	Step 2,000	Step 2,500	Change
P1, future P2–P4	+0.2560	**+0.2908**	+0.0348
P2, future P3–P4	+0.1085	+0.1374	+0.0289
P3, future P4	+0.0193	+0.0260	+0.0067
P4, no future plans	0.0000	0.0000	0.0000

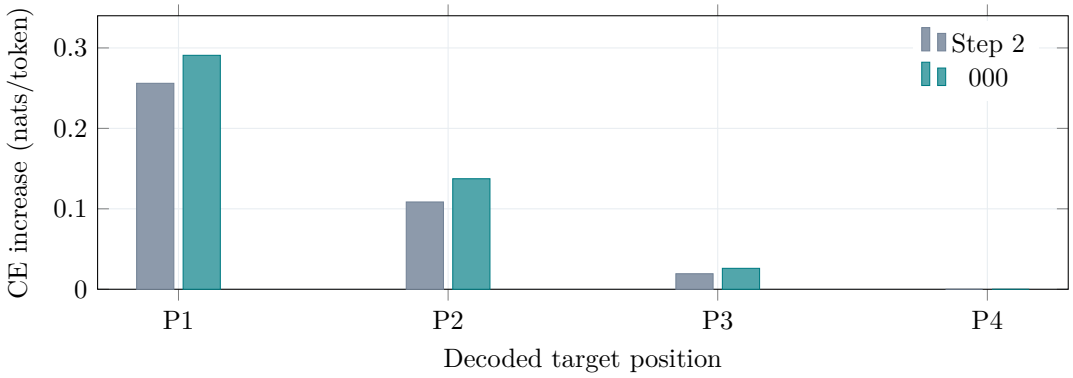


Figure 4: Sensitivity to future PLAN slots persists and grows at P1–P3 in this observed interval. The number of remaining future slots and the number of real preceding tokens both change with the target position.

probability for the correct token in the *intact versus masked frozen model*. This is *not* an estimate of the benefit over a separately trained future-free architecture.

5.4 The persistent earlier soft proposal matters after commitment

At P2 in the step-2,000 run, comparing the own-only and past-plus-current conditions gives

$$\mathrm{CE}_{\mathrm{P2,own}} - \mathrm{CE}_{\mathrm{P2,past+current}} = 0.5785 - 0.1085 = \boxed{0.4700} \text{ nats/token.} \tag{16}$$

Here the actual token y_1 is present through DEC2 in *both* conditions; the extra input in the latter is PLAN1. This suggests a potentially useful dual representation: the decoder can consult both the original soft belief about position one and the hard identity of the token subsequently chosen.

This finding has two important limits. First, $p_1 E$ is a compact expectation rather than a lossless probability vector. Second, neutralizing PLAN1 also changes the contextualized representations of other PLAN positions because those positions attend to one another bidirectionally; the observed CE difference cannot be attributed uniquely to DEC2 directly reading PLAN1 or to the uncertainty of rejected token alternatives. A soft-versus-hard replacement test and a matched retraining control would be needed to isolate those mechanisms.

5.5 Free-running generation: predictive CE and fluency diverge

The evaluation suite generated 64 new tokens per case using two fixed prompts (`math_general_short` and `science_fluids_short`), one seed, greedy and top-k sampling ($k = 40$, temperature 0.8 for sampled output). The step-2,500 outputs remained weak: invented identifiers, incomplete English, domain drift toward code, and self-repetition. For the original $S = 4$ generator, greedy eight-token repetition rates were 0.0526 (math) and 0.1053 (science), while top-k samples were lexically diverse but often unintelligible.

Illustrative excerpt, $S = 4$ at expanded step 2,500 (math greedy):

```
used to the testing of the laborator.
The code is a laborator, the ‘scape‘ function,
the ‘scul‘ function ...
```

This sample does not demonstrate coherent mathematical explanation. Validation CE under teacher forcing must not be equated with generation quality. The small 2,048-token vocabulary and very limited parameter budget complicate broad-domain generation; a cached-generation parity audit and matched AR samples are necessary to separate representational quality from inference implementation effects.

6 Receding-horizon experiment: commitment stride

6.1 Hypothesis

Since later PLAN positions help P1, a plausible modification is to rebuild the entire K-position plan after every committed token and always decode from DEC1. To test this without retraining, patch 0080P4 introduced a configurable *commit stride* $S \in \{1, 2, 4\}$ with $K = 4$:

- $S = 4$: original architecture; one PLAN prefill, four sequential DEC commitments, same plan retained.
- $S = 2$: two DEC commitments, append both real tokens to the main causal KV state, then rebuild the plan.
- $S = 1$: use DEC1 only, append its real committed token, and replan every time.

The *entire* four-horizon proposal is calculated at every replan; unused positions are not accepted. Previous small-decoder PLAN state is discarded at each replan, although committed tokens remain in the main Transformer context. Hence the experiment changes both plan freshness and plan persistence and cannot attribute an outcome uniquely to one of them.

6.2 Paired causal-inference scoring and work

For each stride, a correctness-first generator uses the original causal main-model cache, precomputes all PLAN keys/values, and appends only true earlier tokens when scoring reference continuations. Eight fixed held-out rows with up to 32 targets each yielded exactly 256 scored tokens in this run. The measured losses are *causal-inference-procedure CE*, not identical to the earlier full-window 0080K training validation.

Table 7: Frozen step-2,500 commitment-stride comparison on 256 paired reference targets. Work counts are proposal PLAN prefills, not a measurement of full GPU inference FLOPs or latency.

Stride	CE (nats/token)	Δ CE vs $S = 4$	Full plans	Plans/token
$S = 4$	**3.96349**	0	64	0.250
$S = 2$	4.02332	+0.05983	128	0.500
$S = 1$	4.02431	+0.06082	256	1.000

6.3 Interpretation and free-running comparison

The frozen model trained with complete four-token blocks scored better when it retained its original plan for all four DEC decisions. Replanning every token incurred roughly four times as many full-plan evaluations, with ≈ 0.061 nats/token higher CE on this small sample. The most plausible confounds are: (i) $S = 1$ repeatedly invokes DEC1 in contexts offset from the trained block grid; (ii) replan-and-discard removes earlier soft-plan information that was demonstrably useful to later DEC positions; and (iii) fresh main-model conditioning may not compensate for losing learned causal DEC interactions. A stride-aware retrained model could behave differently.

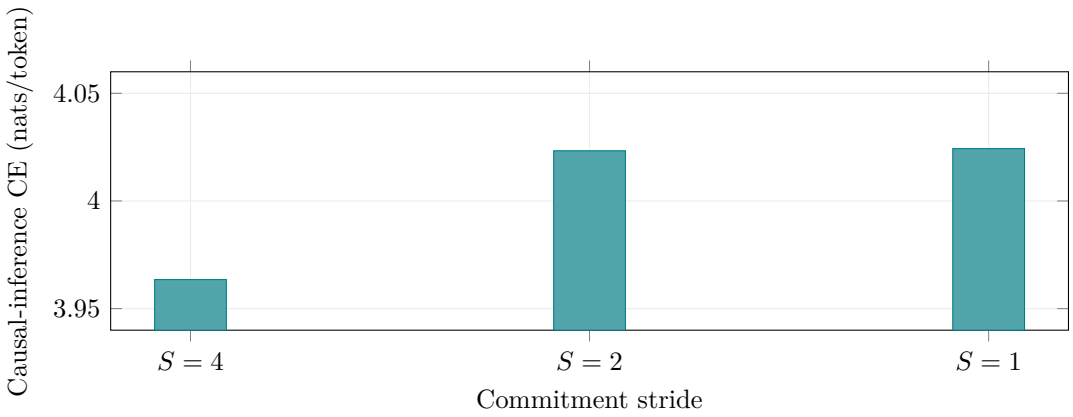


Figure 5: The original $S = 4$ model scores best on this small paired sample, despite performing fewer proposal prefills. The axis is truncated to resolve the observed differences.

Free-running text does *not* show a corresponding clear quality ranking. For example, the two greedy $S = 1$ generations had lower exact eight-gram repetition (0.0175 for math, 0.0000 for science) than the $S = 4$ samples (0.0526 and 0.1053), yet both retained obvious semantic repetition, invented words, and poor relevance to the prompts. Thus fewer repeated eight-grams are not evidence of better overall language quality. Only two prompts and one seed were evaluated; these generation metrics are exploratory.

7 What is established, what is not, and next experiments

7.1 Supported findings

1. A $K = 4$ model with a small dense prefix decoder can be trained end-to-end on packed, document-aware real text with the intended causal block alignment.
2. The correct soft PLAN representations affect held-out teacher-forced predictions even with the same direct proposal-logit prior and shifted ground-truth DEC prefix.
3. A PLAN for a later future position affects earlier decisions, especially P1, whose within-block committed prefix is empty.
4. Earlier soft PLAN slots remain useful to the frozen model after corresponding ground-truth tokens enter its causal DEC prefix.
5. For the *existing* $S = 4$*-trained checkpoint*, inference-only frequent replanning did not improve paired held-out causal CE, despite increasing proposal computations.

7.2 Open questions and threats to validity

1. **Distribution-shifted ablations:** neutral embeddings are outside the trained plan distribution. Context-shuffled realistic proposals, hard-committed-token substitutions, attention tracing, and separately retrained controls are necessary.
2. **Teacher forcing:** later final CE benefits from true preceding tokens, whereas free-running decoding accumulates its own errors. The recorded generation samples remain weak.
3. **No matched AR or multi-token comparison:** superiority over a conventional AR model, comparable auxiliary multi-horizon heads, or a parameter/FLOP-matched prefix decoder is not established.
4. **Small and partly heterogeneous evaluations:** the stride evaluation uses only 256 targets, while other tests use a separate fixed validation regimen. Need document-clustered confidence intervals and more contexts/seeds.
5. **Information content:** a soft expectation pE does not preserve the entire vocabulary distribution.

A measured loss penalty does not prove an explicit sequence-level latent variable, semantic planning, or a new V^K discrete vocabulary.

6. **Scale and implementation:** the CPU experiment does not verify 500M quality or RTX 5080 throughput. Full routed long-context cached generation, objective parity, device peak memory and actual runtime require separate tests.
7. **Long-range effects:** short-block consistency does not automatically solve repetitions or topic drift across many blocks.

7.3 Proposed next experiments

1. **Matched autoregressive baseline:** same tokenizer, corpus, number of distinct supervised target tokens, held-out documents, backbone parameter and compute budgets, plus separately controlled inference time.
2. **Realistic future-plan permutation:** retain P1 but substitute P2–P4 soft vectors from unrelated held-out contexts; examine P1 CE under document-level repeated seeds. Keep residual logits and committed tokens fixed.
3. **Past soft versus committed hard:** at P2 compare original p_1E, the committed embedding $E[y_1]$, a shuffled soft $p_1'E$, and a neutral expectation, keeping the true hard prefix constant; retrain corresponding decoders to address OOD artifacts.
4. **Horizon and decoder-size sweeps:** $K = 2, 4, 8$ and several small decoder capacities with matched target-token counts, seeds and effective compute; plot per-horizon CE and loss penalties at comparable stages.
5. **Stronger stride evaluation:** evaluate many more held-out documents, inspect original-block-position CE and cache parity, then test a stride-aware trained model if evidence warrants the cost.
6. **Free-running robustness:** longer prompt suites, grammar/coherence judgments, repetition onset, semantic repetition, domain adherence, greedy and sampling, and error analysis at K boundaries.
7. **Scaled implementation audit:** on a separate ∼500M backbone, first establish numerical/causal parity, then train and benchmark memory, supervised tok/s, prefill, cached decode, and wall-clock cost.

8 Conclusion

The 0080 architecture is best understood as a *persistent continuous plan conditioned on causal context, resolved by a separate causal commitment stream.* Rather than converting four future marginals into a fixed set of independent token decisions, the decoder retains the original soft predictions throughout the block, jointly contextualizes those predictions with bidirectional PLAN attention, and performs sequential commitments that can consult both previous hard tokens and previous soft beliefs.

Initial 7M-class results show measurable, sometimes growing sensitivity to the complete PLAN pathway and strong first-position dependence on future proposals. These observations motivate investigation of local sequence-level representation, but do *not* yet establish semantic planning, improved generation coherence, or better efficiency than matched existing techniques. The negative frozen commitment-stride experiment is useful: it cautions that repeatedly recomputing the horizon is not automatically preferable to retaining a coherent, shared uncertainty representation. The next stage is discriminating ablation and matched-baseline science, followed by scale-sensitive evaluation.

A Source-code map and reproducibility commands

The following repository paths describe the experimental design. They are implementation references, not a guarantee that every development branch contains identical files. The report is intended to be

committed under a directory such as `docs/research/`.

- `mini_llm/block_proposal/proposals.py`: shared nonlinear horizon proposal head.
- `mini_llm/block_proposal/decoder.py`: soft expectation projection, prefix Transformer, residual correction, manual gradients.
- `mini_llm/block_proposal/contracts.py`: causal block schedule, shifted DEC targets and prefix mask.
- `mini_llm/block_proposal/training.py`: one-trunk forward/backward, bounded independent-block groups.
- `mini_llm/block_proposal/generation.py`: cached soft PLAN prefix and sequential commitment; stride S.
- `examples/train_block_proposal_cpu_learning_0080i.py`: CPU experiment, warm-start and strict resume lineage.
- `examples/evaluate_block_proposal_cross_position_0080k.py`: frozen horizon-specific PLAN controls.
- `examples/evaluate_block_proposal_stride_0080p4.py`: paired causal-inference CE for strides.
- `benchmarks/evaluate_generation.py`: two-prompt, multi-strategy free-running benchmark.

A.1 PowerShell: fixed cross-position ablation

Listing 1: Run against step 2,500 of the expanded-data experiment; from repository root.

```
$env:MINI_LLM_BACKEND = "numpy"
$env:OPENBLAS_NUM_THREADS = "8"
$env:OMP_NUM_THREADS = "8"
.\.venv\Scripts\python.exe -m examples.evaluate_block_proposal_cross_position_0080k `
  --mixed-shard-root .\data\block_proposal_cpu_two_sources_50k `
  --training-output .\benchmarks\results\0080p2_cpu_50k_warmstart `
  --checkpoint .\benchmarks\results\0080p2_cpu_50k_warmstart\checkpoints\two_stage\step_00002500 `
  --output .\benchmarks\results\0080p2_cpu_50k_warmstart\cross_position_step_00002500.json
```

A.2 PowerShell: paired commitment-stride evaluation

Listing 2: The reported small-sample $S \in \{4, 2, 1\}$ likelihood sweep.

```
.\.venv\Scripts\python.exe -m examples.evaluate_block_proposal_stride_0080p4 `
  --mixed-shard-root .\data\block_proposal_cpu_two_sources_50k `
  --training-output .\benchmarks\results\0080p2_cpu_50k_warmstart `
  --checkpoint .\benchmarks\results\0080p2_cpu_50k_warmstart\checkpoints\two_stage\step_00002500 `
  --strides 4,2,1 `
  --validation-rows 8 `
  --target-tokens 32 `
  --output .\benchmarks\results\0080p2_cpu_50k_warmstart\stride_2500_rows8_t32.json
```

A.3 PowerShell: qualitative generation by stride

Listing 3: Same frozen checkpoint and prompts for all three strides.

```
foreach ($stride in @(4, 2, 1)) {
  .\.venv\Scripts\python.exe -m benchmarks.evaluate_generation `
    --architecture block-proposal-0080i `
    --checkpoint ".\benchmarks\results\0080p2_cpu_50k_warmstart\checkpoints\two_stage\step_00002500"
     `
    --tokenizer-path ".\data\block_proposal_cpu_two_sources\tokenizer.json" `
    --commit-stride $stride `
    --output-dir ".\benchmarks\results\0080p4_generation_s${stride}_2500" `
```

```
    --prompt-ids "math_general_short,science_fluids_short" `
    --seeds 42 --strategies "greedy,top-k" `
    --top-k 40 --temperature 0.8 --max-new-tokens 64 `
    --print-generations
}
```

A.4 Run provenance and data availability

The figures and tables were transcribed from user-reported console output for the specific local checkpoints. They were not independently recomputed against the original locally trained checkpoint files in the report-authoring environment. To fully reproduce, preserve checkpoint directories, tokenizers, packed-shard manifests, fixed validation JSONL, experiment output JSON, and generation JSONL. Actual document counts, valid-token totals and hardware timing should be exported from immutable run manifests in a future revision. Do not overwrite the original 5,000-step restricted-corpus run when repeating expanded-data experiments.

B Earlier restricted-corpus K=4 result (not matched)

Table 8: Original restricted-corpus step-5,000 horizon losses, included only for historical context. This used a different prepared dataset and validation draw; do not compare the absolute values directly against the expanded-corpus tables.

Horizon	Proposal CE	Final CE	Gain
P1	4.9904	4.5834	+0.4070
P2	6.0034	4.5850	+1.4184
P3	6.3900	4.6409	+1.7491
P4	6.4153	4.4510	+1.9644

The results initially motivated investigation of cross-position representations, but the strong later-position decoder gains are also naturally influenced by teacher-forced earlier tokens. Historical K=8 experiments used different schedules/data and cannot establish the best K.

References

[1] A. Vaswani et al., *Attention Is All You Need*, Advances in Neural Information Processing Systems, 2017. https://arxiv.org/abs/1706.03762.

[2] L. Dong et al., *Unified Language Model Pre-training for Natural Language Understanding and Generation*, Advances in Neural Information Processing Systems, 2019. https://arxiv.org/abs/1905.03197.

[3] F. Gloeckle, B. Youbi Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve, *Better & Faster Large Language Models via Multi-token Prediction*, 2024. https://arxiv.org/abs/2404.19737.

[4] T. Cai et al., *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*, Proceedings of ICML, 2024. https://proceedings.mlr.press/v235/cai24b.html.